

**ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ**

**ФАКУЛТЕТ „КОМПЮТЪРНИ СИСТЕМИ И  
ТЕХНОЛОГИИ“**



**ДЕМОН ЗА МАРШРУТИЗАЦИЯ ПО  
СЪСТОЯНИЕ НА ВРЪЗКИТЕ И  
ДИСКРЕТНО-СЪБИТИЕН СИМУЛАТОР ЗА  
ИЗСЛЕДВАНЕ НА СХОДИМОСТТА В ТСП/Р  
МРЕЖИ**

**КУРСОВ ПРОЕКТ**

**по дисциплина „Мобилни и стационарни комуникационни мрежи“**

**Разработил:**

Алекс Цветанов, фак. № **[TODO: фак. номер]**

Специалност: Компютърно и софтуерно инженерство, ОКС „Магистър“

**Преподавател:**

Доц. д-р инж. Георги Цочев, Доц. д-р инж. Георги Найденов, Доц. д-р Петко Стоянов

София, **[TODO: година на предаване]**

# СЪДЪРЖАНИЕ

|                                                            |           |
|------------------------------------------------------------|-----------|
| <b>УВОД</b> . . . . .                                      | <b>1</b>  |
| <b>I. ПРЕДМЕТНА ОБЛАСТ И ОБОСНОВКА НА ИЗБОРА</b> . . . . . | <b>2</b>  |
| 1.1. Предметна област . . . . .                            | 2         |
| 1.2. Направени избори и отхвърлени алтернативи . . . . .   | 2         |
| <b>II. ПРОЕКТИРАНЕ И ПРОГРАМНА РЕАЛИЗАЦИЯ</b> . . . . .    | <b>4</b>  |
| 2.1. Структура и абстракция върху времето . . . . .        | 4         |
| 2.2. Протоколно ядро . . . . .                             | 5         |
| 2.3. Симулатор, дневник и възпроизводимост . . . . .       | 5         |
| 2.4. Реализация . . . . .                                  | 6         |
| <b>III. МЕТОДИКА И ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ</b> . . . . . | <b>8</b>  |
| 3.1. Методика . . . . .                                    | 8         |
| 3.2. Резултати . . . . .                                   | 9         |
| <b>ЗАКЛЮЧЕНИЕ</b> . . . . .                                | <b>13</b> |
| <b>ЛИТЕРАТУРА</b> . . . . .                                | <b>14</b> |
| <b>ПРИЛОЖЕНИЕ</b> . . . . .                                | <b>15</b> |

## УВОД

Маршрутизацията в мрежите по стека TCP/IP изглежда решена задача в установено състояние: всеки маршрутизатор поддържа представа за топологията, изчислява най-кратките пътища и попълва маршрутна таблица. Качеството на мрежата обаче се определя от прехода между две установени състояния. При отказ на връзка различните възли научават за него в различен момент и докато представите им се разминават, мрежата образува преходни цикли и губи пакети по пътища, които изглеждат валидни за всеки възел поотделно. Този интервал е времето за сходимост и е предметът на проекта.

Преходът рядко се изучава в лабораторни условия по практическа причина. Отказът в реална мрежа настъпва в момент, който не се повтаря, а таймерите се задвижват от истински часовник. Две наблюдения на едно и също събитие дават различни числа и разликата не може да бъде отдадена на промяна в протокола. За да е измерим, преходът трябва да е възпроизводим, а това изисква времето да се управлява от програмата.

**Целта** е демон за маршрутизация по състояние на връзките от семейството OSPF заедно с дискретно-събитие симулатор, който изпълнява същия програмен код върху управлявано време, и измерване на сходимостта и на преходните цикли при отказ на връзка. **Задачите** са: обосновка на избора; проектиране, при което протоколната логика не зависи от източника на време и от източника на пакети; реализация на откриване на съседи, разпространение на обявленията и изчисляване на маршрути; методика за възпроизводимо измерване; и провеждане на измерванията.

**Обхватът** е маршрутизиращият слой на TCP/IP. Безжичните, клетъчните, спътниковите и АТМ мрежите присъстват като контекст, защото са част от учебното съдържание, но не влизат в реализацията. Причината е методологична: те въвеждат модел на канала със загуби и променлив капацитет и измерената сходимост тогава би зависела от достоверността на модела вместо от поведението на протокола.

**Бележка за предварителните знания на автора.** Работата е предадена преди началото на курса. Авторът е старши софтуерен инженер по C++ и допринася към ngtcp2, реализация на QUIC с отворен код, а в производствена среда работи с механизмите за асинхронен вход и изход на трите платформи: IOCP на Windows, io\_uring на Linux и kqueue на macOS. Границата между протоколната логика и транспорта, върху която стъпва цялото решение, идва оттам.

# **I. ПРЕДМЕТНА ОБЛАСТ И ОБОСНОВКА НА ИЗБОРА**

## **1.1. Предметна област**

Протоколите за маршрутизация вътре в една автономна система се делят на две семейства. Протоколите по вектор на разстоянието обменят със съседите си собствената си маршрутна таблица, като RIP [1]: прости за реализация, но бавни при разпространение на лошите новини и нуждаещи се от отделни механизми срещу цикли. Протоколите по състояние на връзките разпространяват описание на локалната свързаност до всички възли, така че всеки възел изгражда еднакъв граф и изчислява маршрутите самостоятелно. OSPF версия 2 [2] е каноничното описание, а сравнението на двете семейства заедно с причините за преходните цикли е при Perlman [3].

Изчислението се свежда до дърво от най-кратки пътища с корен текущия възел, тоест до алгоритъма на Дийкстра [4]. Изборът на алгоритъм е безспорен, но времето за сходимост не се определя от него: изчислението е бързо спрямо интервалите, през които възлите изобщо научават за промяната. Тези интервали се задават от таймерите за откриване на съсед и от закъснението на лавинното разпространение. Затова литературата за ускоряване на сходимостта се съсредоточава върху по-бързо откриване на отказа, например чрез двупосочното откриване на препращането (BFD) [5]. Измерванията в раздел III потвърждават точно това разпределение.

Управлението на потоците е в друг слой, но е свързано със сходимостта наблюдаемо. Прозорецът за претоварване на TCP се свива при загуба на пакети [6, 7], а преходните цикли произвеждат загуба, която за подателя е неразличима от загуба поради претоварване. Времето за сходимост следователно има пряк ефект върху пропускателната способност. Зрелите симулатори с дискретни събития, OMNeT++ и ns-3, са утвърдена среда за такива изследвания, но симулират собствени модели на протокола; настоящият проект се стреми към друго свойство: една и съща реализация да бъде изпълнима както над симулирано, така и над реално време.

## **1.2. Направени избори и отхвърлени алтернативи**

**Семейство на протокола.** При протокол по вектор на разстоянието всеки възел вижда само обобщени разстояния, така че измерването на преходни цикли изисква външен наблюдател, който събира таблиците на всички възли. При протокол по състояние на

вързките разминаването между представите на два възела е наблюдаемо отвътре. Избран е вторият подход, защото прави измерваната величина видима.

**Модел на времето.** Работа изцяло върху реални интерфейси дава достоверност без повтораемост. Работа изцяло в симулатор дава повтораемост, но проверява модела, а не програмата. Приет е трети вариант: протоколната логика няма достъп до часовник и до сокети, а работи над абстракция, зад която стои или опашка с виртуален часовник, или системният часовник и реалните сокети. Цената е допълнителна граница между слоевете. Ползата е, че измереното поведение принадлежи на реализацията, а не на неин симулационен двойник.

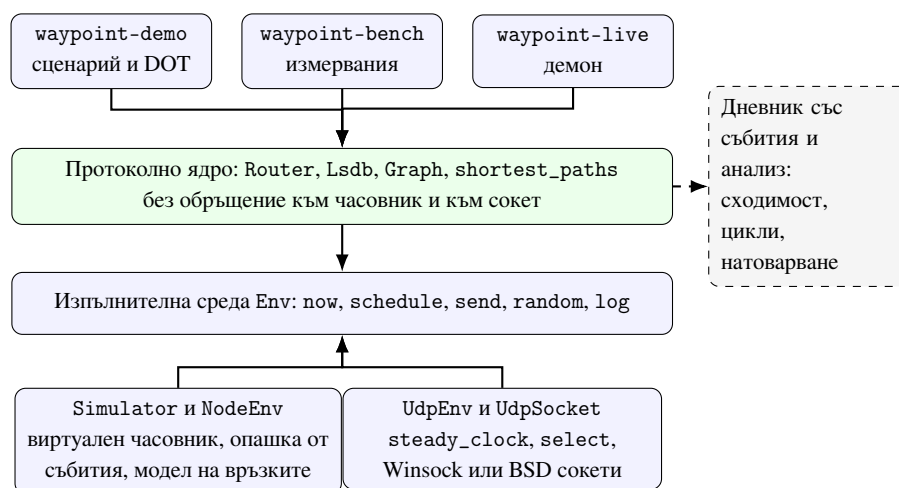
**Език и изграждане.** Разгледани са C++, Rust и Go. Go има среда за изпълнение, която управлява планирането и събирането на паметта и внася закъснения, неотстраними от измерването. Rust дава силни гаранции за безопасност на паметта, но еталонните реализации са на C и интеграцията изисква допълнителен слой. Избран е C++ по стандарта от 2020 година [8]: детерминирано управление на ресурсите и абстракция върху времето, безплатна по време на изпълнение. За изграждане е избран CMake, защото трябва да дава еднакъв резултат върху Windows, Linux и macOS.

**Еталон за коректност.** Като еталон бяха разгледани FRRouting и BIRD. Пълната съвместимост изисква не само форматът по мрежата да е този на OSPFv2 [2], а и транспортът да е IP протокол 89, плюс двойките пакети за обзор с водещ и подчинен, областите и типовете обявления, които обхватът опростява. Форматът по мрежата е реализиран като OSPFv2; жива съседност с FRRouting или BIRD **не е проверена**, защото живият режим носи пакетите по UDP. Вместо външен еталон е използван вътрешен: маршрутните таблици на всички възли се сравняват с таблици, изчислени независимо върху истинската топология. Разликата в силата на двете проверки е отчетена в раздел III, а не премълчана.

## II. ПРОЕКТИРАНЕ И ПРОГРАМНА РЕАЛИЗАЦИЯ

### 2.1. Структура и абстракция върху времето

Системата е на четири слоя (Фиг. 1): изпълнителна среда, протоколно ядро, три изпълними програми и слой за анализ върху маршрутните таблици и дневника със събития.



Фигура 1. Слоевете на системата. Стрелките сочат посоката на зависимостта.

Границата е структурна: заглавните файлове на платформата се включват единствено в `src/udp.c`, така че нарушението ѝ се вижда като нов такъв ред в друг файл. Двата режима изпълняват едни и същи обектни файлове на ядрото. Операция за отмяна на таймер съзнателно липсва, защото би изисквала идентификатор, срок на валидност и правило при отмяна на вече изпълняващ се таймер. Вместо това всеки таймер носи *печат на поколението*, който при изпълнение се сравнява със стойността в структурата на съседа и при несъвпадение извикването се връща без действие: един целочислен инкремент отменя произволен брой висящи таймери, които при виртуално време не струват нищо, докато чакат момента си.

В симулация времето се движи само при изваждане на следващото събитие. В режим на демон източникът е монотонният системен часовник, изпращането е `sendto` по UDP, а изчакването е `select` и на трите платформи: `epoll`, `kqueue` и `WSAPoll` изискват по отделна реализация, а предимството им се проявява при стотици дескриптори, не при един сокет.

## 2.2. Протоколно ядро

Ядрото поддържа таблица със съседите, база с обявленията, индексирана по подател и с поредни номера, и маршрутна таблица от дървото от най-кратки пътища върху графа, реконструиран от базата. Графът е *насочен*, защото всеки маршрутизатор обявява само своята страна на връзката, и преди изчислението се взема двупосочното му подмножество: ребро участва само ако и двата му края са го обявили. Това правило е причината едностранният отказ да води до правилно пренастройване, а не до постоянна черна дупка.

Разделението между база и маршрутна таблица е съществено за измерването: моментът, в който възел научава за промяна, моментът, в който изчислява, и моментът, в който новата таблица влиза в сила, са три различни момента. Инсталирането е отделна стъпка и може да бъде забавено с параметър, защото интервалът между изчислението и влизането в сила е мястото, където възникват преходните цикли.

Изчислението е Дийкстра [4] с 64-битово натрупване, за да не може верига от големи цени да препълни тихо 32-битовата метрика, и с разрешаване на равните разстояния по номер на възела. Многопътността при равна цена се получава чрез сливане на множествата от първи стъпки, чиято коректност почива на строго положителните цени; затова четецът на топологии отхвърля цена нула.

## 2.3. Симулатор, дневник и възпроизводимост

Опашката е подредена по двойката (момент, пореден номер на вписване). Поредният номер е съществен: без него две събития с еднакъв момент биха се изпълнявали в ред, зависещ от контейнера. Всяка връзка се моделира като две независими посоки със закъснение, капацитет и момент, до който посоката е заета, тоест моделът включва опашка и сериализация, не само закъснение. Отказите са четири вида: отказ на връзка в двете посоки, отказ само в едната, отказ на възел и възстановяване.

Всяко значимо събитие се записва като запис от шест полета: момент, възел, вид, отсрещен възел и две числови полета. Анализът работи само върху този дневник и върху маршрутните таблици: сходимостта се проверява чрез сравнение на таблицата на всеки възел с таблица, изчислена независимо върху истинската топология, а преходните цикли чрез проследяване на следващите стъпки за всяка двойка източник и цел. Двете проверки

се извикват след всяко инсталиране, така че моментите се засичат с точността на събитието. Възпроизводимостта се проверява, а не се твърди: дневникът се свежда до 64-битова контролна сума и всеки сценарий се изпълнява два пъти.

## 2.4. Реализация

Исходният текст е 5215 реда: 3158 библиотека, 719 програми и 1338 тестове, разпределени по модули според приложението. Изграждането е с CMake по стандарт C++20 [8]. **Проектът няма зависимости извън стандартната библиотека и системните заглавни файлове за сокети.** Това не е предпочитание по стил: изграждане, което изисква мениджър на пакети или изтегляне по мрежата, не се възпроизвежда на чужда машина. Затова и средството за тестване е собствено, 132 реда макроси в CTest, вместо първоначално предвидения GoogleTest.

Всички цели числа се кодират явно, байт по байт, в мрежов ред (от най-значимия), по формата на OSPFv2 [2]: заглавие от 24 байта, нулева автентикация, контролна сума на пакета по IP (без полето за автентикация) и контролна сума на Fletcher върху всяко обявление. Петте типа пакети, които демонът използва — приветствие, описание на базата, заявка, обновяване и потвърждение — съвпадат с петте типа на OSPFv2. Декодирането проверява обявената дължина и двете контролни суми; обявеният брой обявления се проверява преди заделянето на памет. Златни пакети, генерирани независимо и сверени за контролните суми, са част от тестовете.

Автоматът за съседство е чиста функция от състояние и събитие и следва RFC 2328 [2]. Комбинация извън таблицата оставя състоянието непроменено, което RFC 2328 нарича пренебрегнато събитие. Един от тестовете обхожда всичките петдесет и шест комбинации от седем състояния и осем събития и проверява, че функцията е тотална.

Сравнението на поредните номера на обявленията не е просто  $>$ : номерата се подреждат върху крайно множество, затова се взема знаковата разлика,  $(\text{int32\_t})(a - b) > 0$ , така че обръщането на брояча не спира приемането. Изходът има пет случая: записано, повторение, по-старо, оттеглено и пренебрегнато. Повторението е единственото, което спира лавината.

Живият режим използва същия клас Router с UdpEnv вместо NodeEnv и е проверен с тест върху реални сокети. **Записването на маршрутите в маршрутната таблица на операционната система не е реализирано**, защото не влияе на измерваната величина

и изисква привилегировани интерфейси. **Кодирането по OSPFv2 е реализирано; жива съседност с FRRouting или BIRD не е проверена**, защото тези демони говорят OSPF върху IP протокол 89, а живият режим носи пакетите по UDP.

## III. МЕТОДИКА И ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ

### 3.1. Методика

Измерват се четири величини. **Време за сходимост:** интервалът от отказа до момента, в който маршрутната таблица на всеки останал свързан възел съвпада точно с таблица, изчислена независимо върху истинската топология, като съвпадението обхваща цената, следващите стъпки и предшествениците, а не само достижимостта. **Преходни цикли:** интервали, в които следващите стъпки на два или повече възела образуват затворен път. **Натоварване:** съобщения и байтове след отказа. **Нестабилност:** брой цели, за които някой възел е променил записа си. Времето в симулатора е виртуално, така че натовареността на машината **не влияе на измерените стойности**. Резултатите са получени на хост alex-pc (Microsoft Windows 11 Pro N, OsVersion 10.0.26200; AMD Ryzen 5 3600 6-Core Processor, 12 logical processors) с g++ 15.2.0 MinGW, CMake 4.3.2 и Ninja 1.13.2 в режим Release. Командата от корена на хранилището е `.\build\waypoint-bench --csv results.csv`. Времената в таблиците са милисекунди виртуално време, не стенен часовник.

Топологиите са четири семейства: пръстен, който дава най-голям диаметър и е песимистичният случай; решетка със средна свързаност; пълна мрежа с диаметър единица и максимално натоварване; и дебело дърво, в което пътищата с равна цена са правило. За зависимостта от размера са взети пръстени с 4 до 32 възела, тоест диаметър от 2 до 16; за таймерите пръстен с 12 възела; за плътността четири топологии с по 12 възела и средна степен от 2,00 до 11,00; а за вида на отказа пръстен, решетка четири на четири и дебело дърво с  $k$  равно на четири.

Стойностите по подразбиране са: поддържане 1 s, отпадане 4 s (отношението по RFC 2328 [2]), повторно изпращане 1 s, изчакване преди изчисление 100 ms, закъснение при инсталиране 0, закъснение по връзка 1 ms и капацитет 100 Mbit/s. В експеримента за таймерите поддържането приема от 250 ms до 4 s, а отпадането е четирикратно. В експеримента за циклите всеки маршрутизатор получава закъснение при инсталиране от 0 до 200 ms, изтеглено от зърното, защото записването на нова таблица отнема време. **Ако това закъснение е нула за всички, преходен цикъл е невъзможен по построение**, и измереният нула би бил свойство на модела.

Всяка комбинация се изпълнява по десет пъти със зърна от 1000 до 1009, така че

фазата между последното приветствие и отказа, единственият източник на разсейване при виртуално време, да бъде извадена достатъчно на брой пъти. Всяко изпълнение има загряване от 40 s, отказ на 41-ата секунда и прозорец от 60 s. Резултат е валиден само ако мрежата е била сходима преди отказа, ако е сходима отново в рамките на прозореца и ако повторението със същото зърно дава **идентична контролна сума на дневника**.

### 3.2. Резултати

Изпълнени са 25 комбинации по 10 повторения, тоест 250 резултата, всеки от двойка изпълнения с проверено съвпадение на контролната сума, или общо 500 изпълнения. **Отхвърлени изпълнения: 0**. Всички 250 удовлетворяват и трите условия, а всичките 500 дадоха идентична контролна сума при повторение; за демонстрацията тя е **[TODO: контролна сума след OSPFv2, локално измерване]**. Отделно проверката за начална сходимост беше проведена върху 280 изпълнения на седем пръстена с по 40 зърна, също без отказ; най-бавното начално установяване отне 2,098 s.

**Таблица 1. Време за сходимост след отказ на връзка, пръстен, 10 повторения**

| Топология | Възли | Диаметър | Средно, ms | Най-малко, ms | Най-голямо, ms |
|-----------|-------|----------|------------|---------------|----------------|
| ring4     | 4     | 2        | 3472,7     | 3213,3        | 3743,1         |
| ring6     | 6     | 3        | 3474,7     | 3215,3        | 3745,1         |
| ring8     | 8     | 4        | 3476,7     | 3217,3        | 3747,1         |
| ring12    | 12    | 6        | 3480,7     | 3221,3        | 3751,1         |
| ring16    | 16    | 8        | 3484,7     | 3225,3        | 3755,1         |
| ring24    | 24    | 12       | 3492,7     | 3233,3        | 3763,1         |
| ring32    | 32    | 16       | 3500,7     | 3241,3        | 3771,1         |

**Диаметърът почти не влияе** (Табл. 1). Между диаметър 2 и 16 средното нараства с 28,0 ms, тоест с 2,0 ms на хоп, което съвпада точно със закъснението от 1 ms по връзка, умножено по двата хопа, добавяни от всеки следващ пръстен. Разсейването между повторенията на една и съща топология е 529,8 ms, тоест **деветнадесет пъти по-голямо от целия ефект на диаметъра**. Откриването на отказа отнема почти целия интервал, защото върви по таймера за отпадане от 4 s, докато разпространението отнема милисекунди. Сходимостта се съкращава само чрез по-бързо откриване, което е и причината литературата да се съсредоточава върху механизми като BFD [5].

Зависимостта от таймерите (Табл. 2) е линейна и практически изцяло определена от

**Таблица 2. Време за сходимост спрямо таймерите, пръстен с 12 възела, 10 повторения**

| Поддържане, s | Отпадане, s | Средно, ms | Най-малко | Най-голямо | Отношение |
|---------------|-------------|------------|-----------|------------|-----------|
| 0,25          | 1           | 977,4      | 921,8     | 1033,2     | 0,98      |
| 0,50          | 2           | 1841,5     | 1721,3    | 2030,9     | 0,92      |
| 1,00          | 4           | 3480,7     | 3221,3    | 3751,1     | 0,87      |
| 2,00          | 8           | 6649,9     | 6221,3    | 7511,9     | 0,83      |
| 4,00          | 16          | 13007,7    | 12221,3   | 14351,1    | 0,81      |

интервала за отпадане: средното е между 0,81 и 0,98 от него. Отказът настъпва в момент, независим от фазата на приветствията, така че таймерът за отпадане е бил презареден някъде в предходния интервал за поддържане и откриването отнема отпадането минус тази фаза. Обхватът го потвърждава: разликата между най-голямата и най-малката стойност е 111,4 ms при поддържане 250 ms, 529,8 ms при 1 s и 2129,8 ms при 4 s, тоест от 0,45 до 0,65 от поддържането.

**Таблица 3. Натоварване след отказ на връзка, 12 възела във всички случаи, 10 повторения**

| Топология | Връзки | Степен | Съобщения | Байтове | Промени | Сходимост, ms |
|-----------|--------|--------|-----------|---------|---------|---------------|
| ring12    | 12     | 2,00   | 22,0      | 1408    | 42,0    | 3480,7        |
| mesh2x6   | 16     | 2,67   | 38,0      | 2888    | 54,0    | 3520,5        |
| mesh3x4   | 17     | 2,83   | 42,0      | 3696    | 40,0    | 3421,5        |
| full12    | 66     | 11,00  | 238,0     | 40936   | 2,0     | 3471,7        |

От 12 на 66 връзки, тоест 5,5 пъти, съобщенията растат 10,8 пъти, а байтовете 29,1 пъти (Табл. 3). Всяко обявление се препраща по всяка връзка освен входящата, а байтовете растат още по-бързо, защото обявлението изброява всички съседни на подателя и в пълна мрежа съдържа 11 записа вместо 2. Натоварването расте **квадратично със степента**, което е известното ограничение на протоколите по състояние на връзките и причината реалните разгръщания да делят мрежата на области. Промените в маршрутните таблици вървят в обратната посока, 42 при пръстена и 2 при пълната мрежа: плътността разменя натоварване срещу устойчивост на маршрутите. Сходимостта не следва плътността, и четирите стойности са между 3421 и 3521 ms.

**Топологията определя дали изобщо възниква цикъл, а видът на отказа определя колко често** (Табл. 4). В пръстен цикъл възниква при всяко от тридесетте изпълнения, в решетка четири на четири отказ на връзка не произвежда цикъл в нито едно от двадесетте, а отказ на възел произвежда в три от десетте. Обяснението е в геометрията

**Таблица 4. Преходни цикли по вид на отказа, 10 повторения, разсейване при инсталиране 200 ms**

| Топология | Вид отказ           | Цикли | Продължителност, ms | Сходимост, ms |
|-----------|---------------------|-------|---------------------|---------------|
| ring12    | връзка, двупосочно  | 1,00  | 154,1               | 3710,2        |
| ring12    | връзка, едностранно | 1,00  | 153,4               | 3862,7        |
| ring12    | възел               | 1,00  | 150,7               | 3858,3        |
| mesh4x4   | връзка, двупосочно  | 0,00  | 0,0                 | 3654,6        |
| mesh4x4   | връзка, едностранно | 0,00  | 0,0                 | 3860,5        |
| mesh4x4   | възел               | 0,30  | 19,5                | 3858,9        |
| fattree4  | връзка, двупосочно  | 0,50  | 35,0                | 3605,1        |
| fattree4  | връзка, едностранно | 0,50  | 35,4                | 3726,8        |
| fattree4  | възел               | 1,00  | 65,4                | 3725,7        |

на алтернативния път: в пръстен отказът кара възлите отсреща да препращат в обратна посока и докато единият край на ребро е сменил посоката, а другият не, двата края сочат един към друг. В решетката алтернативният път заобикаля, но не обръща оцеляло ребро, а отказът на възел засяга до четири връзки едновременно и вече може да обърне ребро.

Едностранният отказ дава същия брой цикли, но по-бавна сходимост: 3862,7 срещу 3710,2 ms в пръстена и 3860,5 срещу 3654,6 ms в решетката, тоест от 153 до 206 ms повече, защото при двупосочен отказ и двата края откриват мълчанието по своя таймер, а при едностранен само единият мълчи. Продължителността на циклите, 150,7 до 154,1 ms при разсейване от 200 ms, е от порядъка на самото разсейване, тъй като те се затварят с последната въведена таблица.

Един сценарий е разгледан подробно: пръстен с 12 възела, отказ между възли 6 и 7, зърно 20260819. Възел 6 открива мълчанието пръв, на 3,239 s след отказа, и обявлението му обхожда целия пръстен за 11 ms. Всеки възел изчислява точно 100 ms след като научи, а инсталирането се разстила от 3,369 до 3,539 s. Преходният цикъл, засегнал 24 двойки източник и цел, започва с първата въведена таблица и се затваря с последната, когато мрежата е сходима. Разходът е 22 съобщения и 1408 байта, при 42 променени цели.

**Ограниченията** на измерването са четири. Първо, моделът на връзката няма загуби, така че поведението при влошена, но работеща връзка не е изследвано. Второ, разсейването при инсталиране е параметър на модела, а не измерена стойност от реална апаратура, така че абсолютната продължителност на циклите зависи от избора му; независим от него е фактът кои топологии изобщо произвеждат цикли. Трето, наборът от топологии е малък и правилен, а реалните мрежи не са. Четвърто, **жива съседност**

**с външна реализация на OSPF не е проверена:** форматът по мрежата е OSPFv2, но транспортът е UDP, а FRRouting и BIRD говорят върху IP протокол 89. Вътрешното сверяване проверява, че реализацията изчислява същото, което Дийкстра изчислява върху същия граф, но не че образува съседност с външен демон.

## ЗАКЛЮЧЕНИЕ

Изградени са демон за маршрутизация по състояние на връзките и дискретно-събитийен симулатор, който изпълнява същата протоколна логика върху управлявано време, така че отказът на връзка да бъде възпроизводимо събитие. Протоколното ядро няма достъп до часовника и до сокетите, а работи над абстракция от пет операции, така че едни и същи изходни файлове се изпълняват и в симулация, и над реални интерфейси, и измереното поведение е свойство на реализацията.

**Резултатите** са четири твърдения, всяко от 250 валидни измервания и нула отхвърлени. Първо, времето за сходимост се определя почти изцяло от откриването на отказа: следва интервала за отпадане с отношение от 0,81 до 0,98 и нараства само с 2,0 ms на хоп, деветнадесет пъти по-малко от разсейването между повторенията. Второ, при преминаване от пръстен към пълна мрежа с еднакъв брой възли връзките растат 5,5 пъти, съобщенията 10,8 пъти, а байтовете 29,1 пъти, докато промените в маршрутните таблици намаляват от 42 на 2. Трето, преходните цикли зависят преди всичко от топологията, а признакът е дали алтернативният път обръща оцеляло ребро. Четвърто, едностранният отказ се възстановява от 153 до 206 ms по-бавно от двупосочния.

**Предимствата** спрямо измереното са три: пълна възпроизводимост, потвърдена от 500 изпълнения с идентична контролна сума; достатъчност на абстракцията, тъй като същият клас Router работи и над виртуален часовник, и над реални UDP сокети; и липса на външни зависимости. **Недостатъците** са изброените в раздел III, плюс нереализираното записване на маршрутите в операционната система.

Два дефекта заслужават отбелязване, защото и двата бяха намерени от измерване, а не от разсъждение. Първият беше запазен обзор на базата, който при възпроизвеждане описваше по-ранно състояние и оставяше една връзка невидима за цялата мрежа. Вторият беше заявка за обявление, чийто отговор пристига по-стар от вече наученото от другаде и затова не изчистваше чакането, оставяйки съседството в междинно състояние завинаги. И двата се проявяваха само при определени зърна и щяха да останат незабелязани при неповторимо изпълнение, което е практическото потвърждение на изходната теза. Посоки за по-нататъшна работа са жива съседност с FRRouting или BIRD върху IP протокол 89, записване на маршрутите в операционната система, модел на канал със загуби и по-бързо откриване на отказ [5].

## ЛИТЕРАТУРА

- [1] C. Hedrick, “Routing information protocol,” RFC 1058, Internet Engineering Task Force, June 1988.
- [2] J. Moy, “OSPF version 2,” RFC 2328, Internet Engineering Task Force, April 1998. STD 54.
- [3] R. Perlman, *Interconnections: Bridges, Routers, Switches, and Internetworking Protocols*. Addison-Wesley, 2 ed., 2000.
- [4] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.
- [5] D. Katz and D. Ward, “Bidirectional forwarding detection (BFD),” RFC 5880, Internet Engineering Task Force, June 2010.
- [6] V. Jacobson, “Congestion avoidance and control,” *ACM SIGCOMM Computer Communication Review*, vol. 18, no. 4, pp. 314–329, 1988.
- [7] M. Allman, V. Paxson, and E. Blanton, “TCP congestion control,” RFC 5681, Internet Engineering Task Force, September 2009.
- [8] ISO/IEC, “ISO/IEC 14882:2020 — programming languages — C++,” 2020.

## ПРИЛОЖЕНИЕ. ИЗХОДЕН ТЕКСТ

Пълният изходен текст е в <https://github.com/Alex-Tsvetanov/waypoint> под лиценз MIT. Табл. 5 дава модулите, ролята им и обема им в редове, като заглавният файл и файлът с реализация са събрани.

**Таблица 5. Модули на реализацията, 5215 реда общо**

| Модул      | Съдържание                                                       | Редове |
|------------|------------------------------------------------------------------|--------|
| types      | скаларни типове, номер на възел, момент във времето              | 38     |
| env        | абстракцията от пет операции над време и транспорт               | 67     |
| packet     | OSPFv2 формат по мрежата, кодиране и декодиране с контролни суми | 613    |
| lsdb       | база с обявленията, поредни номера, възраст, оттегляне           | 159    |
| spf        | граф и Дийкстра с многопътност при равна цена                    | 236    |
| neighbor   | автомат за състоянията на съседите като чиста функция            | 160    |
| router     | протоколно ядро: таймери, разпространение, маршрути              | 621    |
| topology   | четец на топологии и генератори за четирите семейства            | 338    |
| sim        | дискретно-събитен симулатор, модел на връзките, откази           | 471    |
| analysis   | сходимост, преходни цикли, натоварване, нестабилност             | 104    |
| udp        | сокети и таймери в жив режим, Winsock и BSD                      | 351    |
| apps/demo  | сценарият с времевата картина и изход в DOT                      | 239    |
| apps/bench | четирите експеримента и записът в CSV                            | 327    |
| apps/live  | демонът, един процес на маршрутизатор                            | 153    |
| tests/     | собствен изпълнител на тестове и осем набора, 76 случая          | 1338   |

Всичките 250 резултата са валидни, така че таблиците в раздел III съдържат пълния набор. Измерванията се възпроизвеждат с `смаке -S . -B build, смаке --build build` и `./build/waypoint-bench --csv results.csv`.